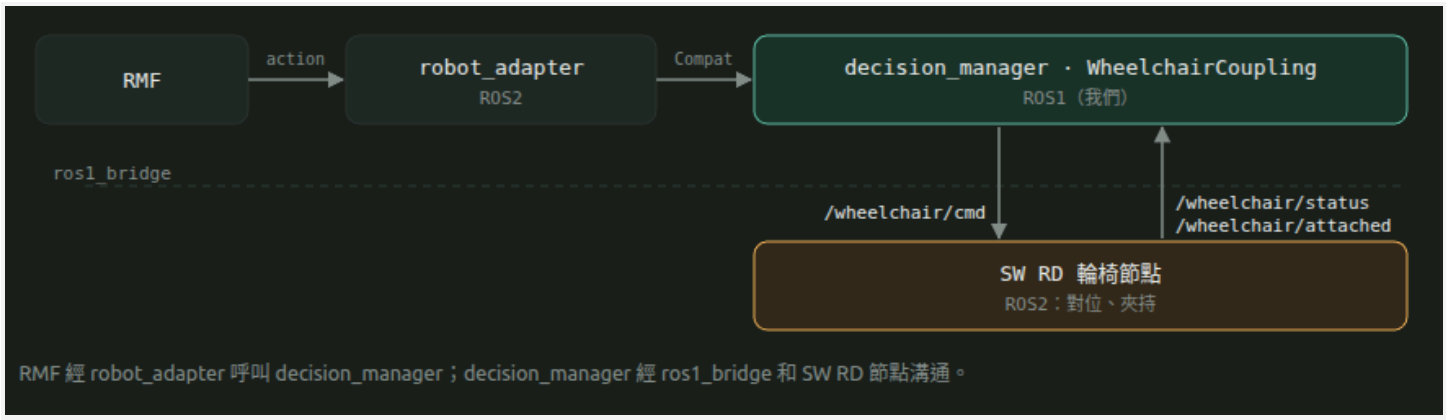
G7+ 輪椅結合／分離 — 介面需求

項目	內容
版本	v0.1
日期	2026-10-12
適用	G7+ service robot（`agv7p_service_robot` ↔ `agv7p_service_robot_wheelchair`）
RMF action	`attach_wheelchair`、`detach_wheelchair`

1. 目的與分工

RMF 派 `attach_wheelchair` 或 `detach_wheelchair` 時，車上的 decision_manager 會執行輪椅任務（暫定名稱 `WheelchairCoupling`）。實際的 **AprilTag** 對位、靠近、夾持、放開由 ROS2 節點負責。Shane透過本文件定義的介面下指令、讀狀態。

負責方	工作內容
Hank（ROS2 節點）	偵測輪椅上的 AprilTag、對位並靠近、控制夾持機構、判斷是否夾好、放開；回報狀態和錯誤
Shane（ROS1 decision_manager）	接收 RMF 指令、確認前置條件、送指令給 Hank、等結果；成功後切換 footprint、重設定位、更新 robot_model；失敗時回報 RMF；取消時送出 cancel
Shane（RMF 和 robot_adapter）	先把車子導航到Wheelchair_Point（準備夾、放輪椅的點），再派 attach 或 detach



2. 設計原則

1. 只用 **`std_msgs`** 和 **`std_srvs`**。車上 ROS1 和 ROS2 之間靠 ros1_bridge 互通。標準型別只要在 `bridge.yaml` 加一筆設定。自訂 msg 要重新編 ros1_bridge 和兩邊的 interface 套件，所以請不要用。
2. 不要用 **ROS2 action**。ros1_bridge 不支援橋接 action。
3. 狀態要持續發布：我們用「多久沒收到狀態」來判斷 ROS2 節點是否還活著。
4. 指令要有編號：每個指令帶 `cmd_id`，狀態要回傳目前處理的是哪一個 `cmd_id`，避免把上一次的結果當成這次的。

3. 介面定義

3.0 通訊方式

所有介面都是 **ROS 2**（**Humble**）原生通訊，Hank 的節點不需要碰 **ROS 1**。Shane這邊由車上的 ros1_bridge 轉接到 ROS 1。

#	介面	ROS 2 通訊類型	Hank 節點的角色	我們的角色
1	`/wheelchair/cmd`	**Topic**（publish / subscribe）	**Subscriber**：`create_subscription<std_msgs::msg::String>`	Publisher
2	`/wheelchair/status`	**Topic**（publish / subscribe）	**Publisher**：`create_publisher<std_msgs::msg::String>`，用 timer 至少 5 Hz 發布	Subscriber
3	`/wheelchair/attached`	**Topic**（publish / subscribe）	**Publisher**：`create_publisher<std_msgs::msg::Bool>`	Subscriber（decision_manager、motion_controller）
4（選用）	`/wheelchair/get_status`	**Service**（request / response）	**Service server**：`create_service<std_srvs::srv::Trigger>`	Client

Hank 單機自測範例：

```
# 模擬送指令
ros2 topic pub --once /wheelchair/cmd std_msgs/msg/String '{data:
"{\"cmd\":\"attach\",\"cmd_id\":1}"}'
# 看狀態
ros2 topic echo /wheelchair/status
ros2 topic echo /wheelchair/attached --qos-durability transient_local --qos-reliability
reliable
# 選用的查詢 service
ros2 service call /wheelchair/get_status std_srvs/srv/Trigger
```

3.0.1 介面一覽

#	名稱	方向	型別	QoS	說明
1	`/wheelchair/cmd`	Shane → Hank	`std_msgs/msg/String`（JSON）	reliable、volatile、depth 10	下指令

2	`/wheelchair/status`	Hank → Shane	`std_msgs/msg/String`（JSON）	reliable、**transient_local**、depth 1	狀態，至少 **5 Hz** 持續發布
3	`/wheelchair/attached`	Hank → Shane	`std_msgs/msg/Bool`	reliable、**transient_local**、depth 1	目前有沒有夾著輪椅；狀態改變時立刻發，平常至少 1 Hz
4（選用）	`/wheelchair/get_status`	Shane → Hank	`std_srvs/srv/Trigger`	–	主動查詢一次；`success=true`，`message` 放和 #2 一樣的 JSON

topic 名稱可以再討論，但定案後就不要改，因為兩邊的 `bridge.yaml` 都要一致。

3.1 指令 `/wheelchair/cmd`

```
{"cmd": "attach", "cmd_id": 17}
```

欄位	型別	說明
`cmd`	string	`attach`、`detach`、`cancel`、`reset` 其中之一
`cmd_id`	int	由Shane每次遞增。`cancel` 帶的是要取消的那個指令的 `cmd_id`

cmd	**Hank 要做的事**	成功時的最終狀態
`attach`	找輪椅上的 AprilTag、對位、靠近、夾持、確認夾好	`ATTACHED`
`detach`	放開夾持機構、確認已放開，再退開回到 Wheelchair_Point（最終狀態 IDLE，中間經過 DETACHED）	`DETACHED`
`cancel`	立即停車，中止目前動作，夾持機構維持安全狀態	`CANCELLED`
`reset`	清除 `ERROR` 或 `CANCELLED`，依實際夾持狀況回到 IDLE 或 ATTACHED	`IDLE / ATTACHED`

3.2 狀態 `/wheelchair/status`

```
{"state": "CLAMPING", "cmd_id": 17, "attached": false,
 "error_code": 0, "error_msg": "", "stamp": 1791355707.10}
```

欄位	型別	說明
`state`	string	見 3.3
`cmd_id`	int	目前或最後處理的指令編號；開機後還沒收到指令時填 0

attached	bool	和 /wheelchair/attached 相同
error_code	int	見 3.4；正常時填 0
error_msg	string	給人看的錯誤說明，會轉給 RMF 顯示（英文或中文皆可）
stamp	float	發布時間（epoch 秒），用來判斷狀態是否過期

3.3 狀態機

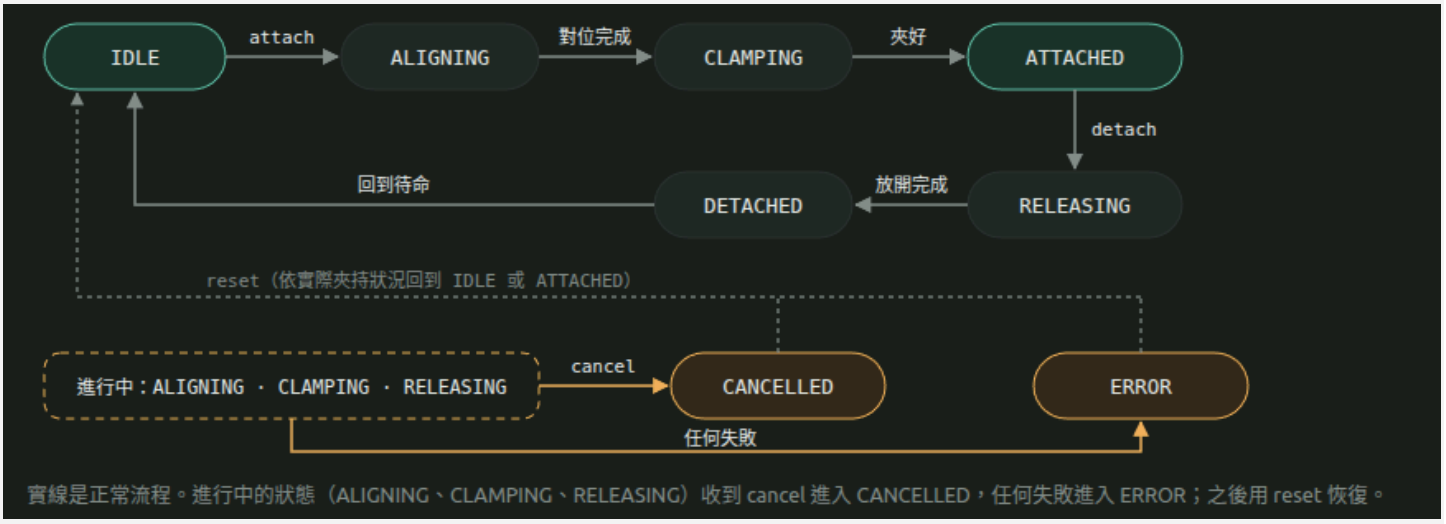
使用流程：

階段	RMF／車子	輪椅節點狀態	robot_model
沒有輪椅	車子在地圖上任一點待命，或執行一般任務	IDLE	agv7p_service_robot
夾輪椅	RMF 先導航到 Wheelchair_Point，再派 attach_wheelchair	IDLE → ALIGNING → CLAMPING → ATTACHED	夾好（attached=true）後切成 agv7p_service_robot_wheelchair
帶著輪椅	RMF 派車接送人員等	ATTACHED	agv7p_service_robot_wheelchair
放輪椅	RMF 先導航到 Wheelchair_Point，再派 detach_wheelchair	ATTACHED → RELEASING → DETACHED（退開中）→ IDLE（已回到 Wheelchair_Point）	放開（attached=false）後切回 agv7p_service_robot
放開之後	RMF 可以派車去充電等	IDLE	agv7p_service_robot

完成判斷：長時間停留的狀態只有 IDLE（attached=false）和 ATTACHED（attached=true）。attach 看到 ATTACHED 且 cmd_id 相符就算完成；detach 要等到 IDLE 且 cmd_id 相符（也就是已退回 Wheelchair_Point）才算完成，DETACHED 只代表已放開、還在退開。

狀態：

state	意思
`IDLE`	沒有夾著輪椅，待命。開機時沒夾著輪椅也是 `IDLE`；detach 完成時車子在 `Wheelchair_Point`
`ALIGNING`	找 tag、對位、靠近中
`CLAMPING`	夾持中
`ATTACHED`	已夾好（attach 成功），帶著輪椅時一直維持這個狀態
`RELEASING`	放開中
`DETACHED`	已放開（`attached=false`），正在退開回 `Wheelchair_Point`。退到位後自行轉為 `IDLE`
`CANCELLED`	被 cancel 中止，車子已停
`ERROR`	失敗，看 `error_code` 和 `error_msg`



3.4 錯誤碼（Hank 可以擴充，但這幾個請保留）

code	意思
0	正常
1	找不到 AprilTag，或 tag 位置無效
2	起始位置超出可對位範圍（太遠或角度太大）
3	對位逾時
4	遇到障礙物而停止
5	夾持機構故障或動作逾時
6	夾持後檢查失敗（沒夾到）

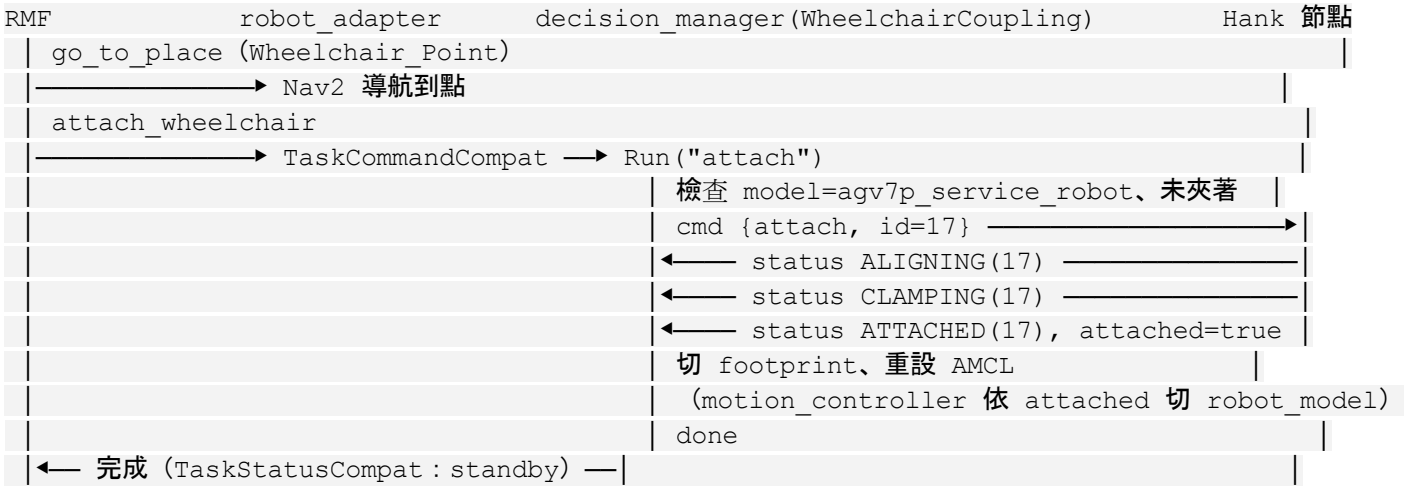
7	放開失敗
8	狀態不符（例如已經夾著又收到 attach）
9	硬體或通訊錯誤

4. 行為要求

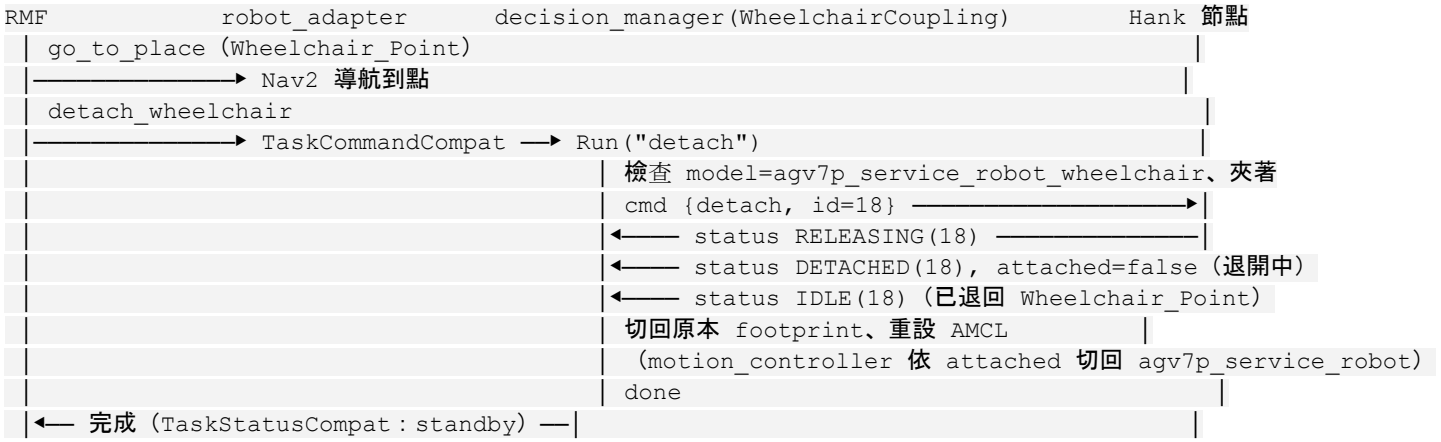
#	要求	原因
R1	收到 `cancel` 後 **0.5** 秒內停車，並回報 `CANCELLED`	RMF 取消、按暫停、`ClearAllTasks` 最後都會走到這裡。如果停不下來，就無法從外部中止
R2	每個階段都要有自己的逾時，超時就進入 `ERROR`（錯誤碼 3 或 5），不能停在 `ALIGNING` 或 `CLAMPING` 不動	我們也會設一個總逾時，但只能當保險
R3	`/wheelchair/status` 要持續發布，至少 5 Hz	我們用超過 1 秒沒收到來判斷節點異常（和現有夾爪 `ex_motor_controller` 的作法一致）
R4	重複指令要處理：已經是 `ATTACHED` 又收到 `attach`，直接回報 `ATTACHED`，或回報 `ERROR` 錯誤碼 8，不能再夾一次；detach 也一樣	避免重送指令造成危險動作
R5	`attached` 要以實際感測為準，不能只看指令有沒有執行	motion_controller 依這個值切換 robot_model，RMF 也依 robot_model 決定可以派哪個 action
R6	開機或重啟後要立刻發布目前的實際狀態（transient_local，或提供 #4 service）	decision_manager 重啟後要知道車上是不是夾著輪椅
R7	移動底盤時要遵守車上的安全機制：急停、雷射安全區；要說明速度發到哪個 topic（`/cmd_vel` 或其他）、最大速度、有沒有自己的避障	Hank 節點會直接控制車子，安全責任要說清楚
R8	只在收到指令時才移動或動作夾持機構，平常不可以自己動	避免和 Nav2、其他任務搶控制
R9	對位用的相機和 AprilTag 不能和現有功能衝突：充電站用 front_camera、tag ID 107（36h11、7.2 cm）	避免辨識錯誤或相機被佔用

5. 時序

5.1 attach 成功



5.2 detach 成功



5.3 失敗

Hank 回報 `ERROR`, 帶錯誤碼和 `error_msg` → 我們回報 fail, `error_msg` 轉給 RMF → RMF 顯示失敗原因。之後我們會送 `reset`。

5.4 取消

RMF 取消, 或按暫停 → decision_manager 呼叫 `Stop()` → 我們送 `cancel` → Hank 停車並回報 `CANCELLED` → 我們回報 stop。

6. 待確認資料

#	項目	用在哪裡
D1	開始對位的條件？	決定 RMF 先導航到哪個點：Wheelchair_Point **Wheelchair_Point** 要設在輪椅前方多遠、角度容許多少，我們依此在地圖上設定這個點
D2	夾著輪椅後的外形尺寸（長、寬，或多邊形）	在 Nav2 新增「帶著輪椅」的 footprint
D3	結合後底盤中心的位移量	決定 TF 要不要位移、AMCL 位置要不要重設（推車是 0.569 m 和 0.625 m）
D4	用哪顆相機、AprilTag 的家族、ID、尺寸	確認不和現有功能衝突（R9）
D5	移動時是否需要我們關閉 AMCL；結束時車子的位置精度	我們任務的前後處理
D6	速度輸出的 topic、最大速度和加速度、避障方式	安全確認（R7）
D7	detach 包含退開，退回 Wheelchair_Point。請說明退開的方式（距離、方向、速度，怎麼確認已回到 Wheelchair_Point）	決定 detach 後車子的位置，以及 RMF 下一段路徑從哪裡開始：Wheelchair_Point
D8	節點名稱、套件和 launch 檔、跑在哪個容器、要不要開機自動啟動	部署
D9	單機測試方式：不經過 decision_manager，直接用 `ros2 topic pub` 測試的範例指令	雙方各自驗證

7. 待討論事項

- [ ] topic 名稱和 namespace 是否定案為 `/wheelchair/*`。
- [ ] 測試時程：Hank 先提供可以單獨測試的版本（D9），我們再接 decision_manager 和 RMF。

附錄：我們這邊的實作

- **decision_manager** 新任務 **`WheelchairCoupling`**：參數 `attach` 或 `detach`。
- 送出指令後，attach 等 ATTACHED、detach 等 IDLE（已退回 Wheelchair_Point），或等到 ERROR、總逾時。
- 成功後切換 footprint、重設 AMCL。
- 失敗時把 `error_msg` 回報給 RMF。
- `Stop()` 時送出 `cancel`。
- **motion_controller**：依 `/wheelchair/attached` 把 robot_model 切換成 `agv7p_service_robot` 或 `agv7p_service_robot_wheelchair`。
- **`bridge.yaml`**：ros_bridge 加上第 3 節的 4 個介面。
- **robot_adapter**：`attach_wheelchair` 和 `detach_wheelchair` 對應到 `WheelchairCoupling`，參數 `attach` 和 `detach`。